

Laboratorio 11 · mongodump, copia binaria, snapshot con fsyncLock y hot backup

Duración: 25 minutos · Carpeta de trabajo: entorno/

Objetivos

Vas a practicar las cuatro familias de backup nativas: el volcado lógico con `mongodump`, la copia binaria en frío de un secundario, la copia "tipo snapshot" congelando las escrituras con `fsyncLock` y el *hot backup* de Percona Server. Y, lo más importante, vas a **restaurar**: un backup que no se ha probado a restaurar no es un backup.

Preparación

```
cd entorno
docker compose up -d
docker compose exec toolbox bash /scripts/estado.sh
```

Si el replica set no está sano o faltan datos, reconstrúyelo con el atajo del laboratorio 05.

Parte A · Backup lógico con mongodump

```
docker compose exec toolbox bash
mongodump --uri "$RS" --readPreference=secondary --oplog --gzip --
archive=/dumps/completo.archive.gz
ls -lh /dumps
```

`--oplog` captura también las operaciones que ocurren **mientras** dura el volcado, para obtener una copia coherente en un único instante. Ahora simula un error humano y restaura solo lo necesario:

```
mongosh "$RS" --quiet --eval 'db.getSiblingDB("tienda").clientes.drop()'
mongosh "$RS" --quiet --eval
'db.getSiblingDB("tienda").getCollectionNames()'
mongorestore --uri "$RS" --gzip --archive=/dumps/completo.archive.gz --
nsInclude="tienda.clientes"
```

```
mongosh "$RS" --quiet --eval  
'db.getSiblingDB("tienda").clientes.countDocuments()'
```

Y una restauración a otro espacio de nombres, útil para comparar datos sin pisar los de producción:

```
mongorestore --uri "$RS" --gzip --archive=/dumps/completo.archive.gz --  
nsInclude="tienda.productos" --nsFrom="tienda.productos" --  
nsTo="restaurado.productos"  
mongosh "$RS" --quiet --eval  
'db.getSiblingDB("restaurado").productos.countDocuments()'  
exit
```

▼ ¿Por qué no hemos usado --oplogReplay en la restauración parcial?

--oplogReplay aplica las entradas del oplog capturadas durante el volcado y solo tiene sentido al restaurar el volcado **completo**: si restauras una sola colección, las operaciones del oplog sobre otras colecciones no tendrían dónde aplicarse de forma coherente. En una restauración completa sí debes usarlo para obtener el estado exacto del final del volcado. Recuerda además que mongodump no copia los índices como datos sino como definiciones: se reconstruyen al restaurar, lo que en colecciones grandes lleva mucho tiempo.

Parte B · Copia binaria en frío de un secundario

Elige un secundario (en el ejemplo, `mongo3`), páralo limpiamente y copia su volumen con un contenedor auxiliar:

```
docker compose exec toolbox bash /scripts/estado.sh  
docker compose stop mongo3  
docker run --rm -v curso-mongo_mongo3-data:/data:ro -v curso-  
mongo_dumps:/backup mongo:8.0 tar czf /backup/mongo3-frio.tgz -C /data .  
docker compose start mongo3  
docker compose exec toolbox bash /scripts/estado.sh
```

Durante la copia el replica set ha seguido funcionando con dos nodos. Ahora **prueba la restauración** levantando un servidor independiente a partir de ese fichero:

```
docker volume create verificacion-data  
docker run --rm -v verificacion-data:/data -v curso-mongo_dumps:/backup  
mongo:8.0 tar xzf /backup/mongo3-frio.tgz -C /data  
docker run -d --name verificacion -v verificacion-data:/data/db
```

```
mongodb/mongodb-community-server:8.0-ubi9
docker exec -it verificacion mongosh
```

Dentro de `mongosh` (si da error de conexión, espera unos segundos a que termine de arrancar):

```
> db.getSiblingDB("tienda").pedidos.countDocuments()
> db.getSiblingDB("admin").system.users.find({}, { user: 1 }).toArray()
> exit
```

```
docker rm -f verificacion
docker volume rm verificacion-data
```

¿Te llama la atención algo del último comando?

▼ Ver solución

El servidor de verificación arranca en modo standalone y **sin autenticación**, así que cualquiera puede leer todos los datos, incluida la colección de usuarios (con sus claves SCRAM derivadas). Un backup contiene todo lo que hay en la base de datos y no hereda la configuración de seguridad del servidor: los backups se tienen que cifrar y custodiar con el mismo cuidado que la producción. Fíjate también en que el nodo ha arrancado aunque sus datos pertenecen a un replica set: en modo standalone ignora la configuración de replicación, que es justo lo que se hace para restauraciones o tareas de mantenimiento.

Parte C · Copia en caliente con fsyncLock (patrón snapshot)

Los snapshots del sistema de ficheros (LVM, ZFS, snapshots de disco en la nube) son la forma más rápida de hacer backup de volúmenes grandes. Si el journal está en el mismo volumen que los datos, el snapshot ya es coherente; si no, o si hay dudas, se congelan las escrituras del nodo durante el instante del snapshot. Vamos a simularlo con `tar` sobre un secundario **sin pararlo**:

```
docker compose exec toolbox bash /scripts/estado.sh
docker compose exec toolbox bash /scripts/fsync.sh lock mongo2
docker run --rm -v curso-mongo_mongo2-data:/data:ro -v curso-
mongo_dumps:/backup mongo:8.0 tar czf /backup/mongo2-snapshot.tgz -C
/data .
docker compose exec toolbox bash /scripts/estado.sh
docker compose exec toolbox bash /scripts/fsync.sh unlock mongo2
docker compose exec toolbox bash /scripts/estado.sh
```

(Si `mongo2` es el primario, usa un secundario: **nunca** bloquee el primario en producción, porque bloquearías todas las escrituras de la aplicación.)

Mientras el nodo estaba bloqueado, ¿qué le ha pasado a su lag de replicación? ¿Por qué el `fsyncUnlock` es tan crítico en un script de backup?

▼ Ver solución

Mientras está bloqueado, el secundario no aplica el oplog, así que su lag crece. Si el script de backup falla después del `fsyncLock` y nunca ejecuta el `fsyncUnlock`, el nodo se queda bloqueado indefinidamente, su lag no para de crecer y, si supera la ventana del oplog, necesitará una resincronización completa. Por eso los scripts de snapshot deben tener un `trap` o un bloque `finally` que garantice el desbloqueo. `fsyncLock` es además acumulativo: cada llamada incrementa un contador y hacen falta tantos `fsyncUnlock` como `fsyncLock` se hayan hecho.

Parte D · Hot backup con Percona Server (opcional)

Percona Server for MongoDB incluye el comando `createBackup`, que hace una copia física coherente de los ficheros de datos **con el servidor en marcha y sin bloquear escrituras**. Usamos el entorno del laboratorio 10:

```
cd ../dia3/lab10-auditoria-cifrado
docker compose up -d
docker compose exec toolbox bash
mongosh "$PS" --quiet --eval 'printjson(db.adminCommand({ createBackup:
1, backupDir: "/hotbackup/copia1" })))'
exit
docker compose exec psmdb ls -l /hotbackup/copia1
docker compose stop
cd ../../entorno
```

La copia contiene los ficheros de WiredTiger listos para usar como `dbPath` de un nuevo servidor. Como el servidor tiene cifrado en reposo, la copia también está cifrada: para restaurarla hace falta **la misma clave maestra**.

Resumen

Método	Tipo	Coherencia	Ventajas	Inconvenientes
<code>mongodump</code>	Lógico	Con <code>--oplog</code>	Granular, portable entre versiones	Lento en volúmenes grandes, índices se reconstruyen
Copia en frío	Físico	Total	Sencilla, restauración rápida	Requiere parar un nodo
Snapshot + <code>fsyncLock</code>	Físico	Total	Muy rápido con LVM/cloud	Depende de la infraestructura; riesgo de

Método	Tipo	Coherencia	Ventajas	Inconvenientes
				dejar el nodo bloqueado
<code>createBackup</code> (PSMDB)	Físico	Total	En caliente, sin bloqueo	Solo Percona Server; copia de un nodo
PBM	Lógico / físico	Consistente en cluster + PITR	Automatizable, PITR, sharding	Una pieza más a operar (siguiente laboratorio)